

6 - OOP

May 19, 2026

Lập trình hướng đối tượng (OOP - Object - Oriented Programming)

TS. Lê Thành Trung

May 19, 2026

Contents

1	Mục tiêu bài học	3
2	Giới thiệu về OOP	3
2.1	OOP là gì?	3
2.2	Khi nào nên dùng OOP?	3
2.3	Lỗi thường gặp ở phần mở đầu OOP	4
3	Lớp và Đối tượng (Class and Object)	4
3.1	Lớp (Class)	4
3.2	Đối tượng (Object)	5
3.3	Thuộc tính lớp và thuộc tính đối tượng (Class vs Instance Attributes)	5
3.4	Lỗi thường gặp	6
4	Tổng kết	6
5	Bài tập	7
5.1	Bài 1	7
5.2	Bài 2	7
5.3	Bài 3	7
5.4	Bài 4	7

1 Mục tiêu bài học

- Hiểu tư duy lập trình hướng đối tượng (Object-Oriented Programming - OOP) và khi nào nên dùng.
- Phân biệt lớp (class) và đối tượng (object).
- Viết được lớp (class) cơ bản với hàm khởi tạo (constructor), phương thức (method), thuộc tính (attribute).
- Phân biệt thuộc tính lớp (class attribute) và thuộc tính đối tượng (instance attribute).

2 Giới thiệu về OOP

2.1 OOP là gì?

Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là cách tổ chức chương trình xoay quanh **đối tượng** (object), trong đó mỗi đối tượng gồm: - Dữ liệu (thuộc tính - attributes) - Hành vi (phương thức - methods)

Tư duy chính: - Gom dữ liệu và hành vi liên quan vào cùng một nơi. - Mô hình hóa bài toán gần với thế giới thực. - Dễ mở rộng, bảo trì khi hệ thống lớn dần.

2.2 Khi nào nên dùng OOP?

- Khi chương trình có nhiều thực thể giống nhau (sinh viên, tài khoản, sản phẩm...).
- Khi cần tái sử dụng logic cho nhiều đối tượng.
- Khi muốn mã nguồn (code) rõ ràng, dễ chia nhóm làm việc.

```
[2]: # Ví dụ: So sánh cách viết không OOP và OOP

# Không OOP: dữ liệu và thao tác rời rạc
student_name = "An"
student_scores = [8, 9, 7]

def calc_avg(scores):
    return sum(scores) / len(scores)

print("[Không OOP]", student_name, "- Điểm TB:",
      round(calc_avg(student_scores), 2))

# OOP: gom dữ liệu + hành vi vào class
class Student:
    def __init__(self, name, scores):
        self.name = name
        self.scores = scores

    def average(self):
        return sum(self.scores) / len(self.scores)

an = Student("An", [8, 9, 7])
```

```
print("[OOP]", an.name, "- Điểm TB:", round(an.average(), 2))
```

[Không OOP] An - Điểm TB: 8.0

[OOP] An - Điểm TB: 8.0

2.3 Lỗi thường gặp ở phần mở đầu OOP

- Nhầm lẫn lớp (class) với đối tượng (object): class là bản thiết kế, object là thực thể cụ thể.
- Viết quá nhiều biến rời rạc thay vì gom vào đối tượng.
- Dùng OOP cho bài toán quá nhỏ, khiến code dài không cần thiết.

3 Lớp và Đối tượng (Class and Object)

3.1 Lớp (Class)

Lớp (class) là khuôn mẫu để tạo đối tượng (object).

Một class cơ bản thường có: - `__init__`: hàm khởi tạo đối tượng (constructor). - Thuộc tính (attributes): nơi lưu dữ liệu. - Phương thức (methods): hàm mô tả hành vi của đối tượng.

```
[ ]: class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

r1 = Rectangle(5, 3)
print("Dien tich:", r1.area())
print("Chu vi:", r1.perimeter())
```

Dien tich: 15

Chu vi: 16

5 3

```
[15]: class SinhVien:
    ten_khoa = "Toán - Thống kê" # thuộc tính lớp (class attribute)

    def __init__(self, ten, tuoi):
        self.ten = ten
        self.tuoi = tuoi

    def thong_tin(self):
        return f"Sinh vien: {self.ten}, Tuoi: {self.tuoi}, Khoa: {self.ten_khoa}"
```

```
sv1 = SinhVien("An", 20)
print(sv1.thong_tin())
```

Sinh vien: An, Tui: 20, Khoa: Toán - Thống kê

3.2 Đối tượng (Object)

Đối tượng (object) là một thực thể cụ thể được tạo ra từ lớp (class).

Ví dụ: từ class Rectangle, ta có thể tạo nhiều object khác nhau với dữ liệu riêng.

```
[6]: # Ví dụ: Tạo nhiều object từ cùng một class
r2 = Rectangle(10, 4)
r3 = Rectangle(7, 6)

print("r2 - Diện tích:", r2.area(), "| Chu vi:", r2.perimeter())
print("r3 - Diện tích:", r3.area(), "| Chu vi:", r3.perimeter())

print("r2 và r3 là 2 đối tượng khác nhau:", r2 is not r3)
```

```
r2 - Diện tích: 40 | Chu vi: 28
r3 - Diện tích: 42 | Chu vi: 26
r2 và r3 là 2 đối tượng khác nhau: True
```

3.3 Thuộc tính lớp và thuộc tính đối tượng (Class vs Instance Attributes)

- **Thuộc tính lớp (class attribute):** thuộc về class, dùng chung cho mọi object.
- **Thuộc tính đối tượng (instance attribute):** thuộc về từng object, mỗi object có giá trị riêng.

Hiểu nhầm phần này là lỗi rất phổ biến khi mới học OOP.

```
[8]: # Ví dụ: Thuộc tính lớp (class attribute) và thuộc tính đối tượng (instance
    ↪ attribute)
class BankAccount:
    bank_name = "ABC Bank" # thuộc tính lớp (class attribute - dùng chung)

    def __init__(self, owner, balance=0):
        self.owner = owner # thuộc tính đối tượng (instance attribute)
        self.balance = balance # thuộc tính đối tượng (instance attribute)

    def deposit(self, amount):
        self.balance += amount

    def info(self):
        return f"{self.owner} | {self.bank_name} | Số dư: {self.balance}"

acc1 = BankAccount("Lan", 1000)
```

```
acc2 = BankAccount("Binh", 500)

acc1.deposit(250)
print(acc1.info())
print(acc2.info())

# Đối thuộc tính lớp từ chính class
BankAccount.bank_name = "XYZ Bank"
print(acc1.info())
print(acc2.info())
```

```
Lan | ABC Bank | So du: 1250
Binh | ABC Bank | So du: 500
Lan | XYZ Bank | So du: 1250
Binh | XYZ Bank | So du: 500
```

[9]: *# Tình huống dễ nhầm: gán qua object sẽ tạo thuộc tính đối tượng mới (instance_ ↪ attribute)*

```
acc1.bank_name = "Ten riêng cho Lan" # KHÔNG đối thuộc tính lớp, chỉ đối riêng ↪ acc1

print("acc1.bank_name:", acc1.bank_name)
print("acc2.bank_name:", acc2.bank_name)
print("BankAccount.bank_name:", BankAccount.bank_name)
```

```
acc1.bank_name: Ten riêng cho Lan
acc2.bank_name: XYZ Bank
BankAccount.bank_name: XYZ Bank
```

3.4 Lỗi thường gặp

- Quên tham số `self` trong phương thức (method).
- Nhầm `ClassName.attribute` với `object.attribute`.
- Gán trực tiếp nhầm chỗ làm thay đổi dữ liệu không như mong đợi.
- Hàm khởi tạo (constructor) nhận dữ liệu nhưng không gán vào `self`.

4 Tổng kết

- OOP giúp tổ chức chương trình theo object gồm dữ liệu + hành vi.
- Lớp (class) là khuôn mẫu, đối tượng (object) là thực thể cụ thể.
- Hàm khởi tạo (constructor) `__init__` dùng để khởi tạo dữ liệu ban đầu.
- Cần phân biệt thuộc tính lớp (class attribute) và thuộc tính đối tượng (instance attribute) để tránh lỗi (bug).

Danh sách tự kiểm tra (checklist): - Bạn đã tạo được class với ít nhất 2 phương thức (methods) chưa? - Bạn có thể tạo nhiều object độc lập từ cùng một class chưa? - Bạn có giải thích được vì sao `obj.attr = ...` có thể không làm đổi `class.attr` không?

5 Bài tập

5.1 Bài 1

Tạo lớp (class) **Book** có các thuộc tính (attributes): - **title**, **author**, **price**

Viết phương thức (methods): - **display_info()** in thông tin sách - **discount(percent)** giảm giá theo phần trăm

5.2 Bài 2

Tạo lớp (class) **Student** có: - **name**, **scores** (danh sách điểm - list)

Viết phương thức (methods): - **average()** trả về điểm trung bình - **classify()** trả về “Giỏi”, “Kha”, “Trung bình”, “Yếu” theo điểm TB

5.3 Bài 3

Tạo lớp (class) **Employee**: - Thuộc tính lớp (class attribute): **company = "ABC Tech"** - Thuộc tính đối tượng (instance attributes): **name**, **salary**

Yêu cầu: - Tạo 3 nhân viên - Đổi **company** từ class và quan sát toàn bộ object - Gán **company** cho riêng 1 object và giải thích kết quả

5.4 Bài 4

Tạo lớp (class) **ShoppingCart**: - Thuộc tính: **items** (danh sách bộ giá trị - tuple, dạng (**ten_sp**, **so_luong**, **don_gia**))

Phương thức (methods): - **add_item(name, qty, price)** - **total()** tính tổng tiền - **print_bill()** in hóa đơn đơn giản

Gợi ý mở rộng: - Thêm VAT 8% - Thêm giảm giá nếu tổng tiền vượt ngưỡng